

Problem 1: Dynamic Max-Flow Update

1. Analisi dei Casi Sia $\hat{e} = (u, v)$ l'arco la cui capacità viene ridotta di un'unità. Consideriamo due scenari in base al flusso originale $f(\hat{e})$: * **Caso 1** ($f(u, v) < c(u, v)$): L'arco non era saturo. La riduzione di capacità non invalida il flusso corrente, poiché $f(u, v) \leq c(u, v) - 1 = \hat{c}(u, v)$. Il flusso f rimane ammissibile e il suo valore massimo non cambia. L'algoritmo termina senza fare nulla. * **Caso 2** ($f(u, v) = c(u, v)$): L'arco era saturo. La nuova capacità $\hat{c}(u, v) = f(u, v) - 1$ rende il flusso corrente non ammissibile. È strettamente necessario rimuovere un'unità di flusso da $\hat{e}$, generando così un eccesso di flusso pari a $+1$ nel nodo u e un deficit pari a -1 nel nodo v .

2. Algoritmo di Riparazione (Caso 2) Costruiamo il grafo residuo G_f basato sulle nuove capacità e cerchiamo di ripristinare la conservazione del flusso: 1. Riduciamo forzatamente $f(u, v)$ di 1. 2. Eseguiamo una visita (BFS o DFS) per cercare un cammino semplice P da u a v all'interno di G_f . * *Se il cammino esiste*: Spingiamo 1 unità di flusso lungo P . Questo sposta l'eccesso da u fino a v , compensando il deficit. Il valore totale del flusso massimo della rete rimane inalterato e l'algoritmo termina. 3. *Se il cammino non esiste*: Significa che il taglio minimo è cambiato e il flusso massimo della rete deve obbligatoriamente scendere di 1. Dobbiamo annullare il flusso in eccesso: * Troviamo un cammino P_1 da u a s in G_f e vi spingiamo 1 unità di flusso (questo cancella 1 unità che in origine era stata inviata dalla sorgente a u). * Troviamo un cammino P_2 da t a v in G_f e vi spingiamo 1 unità di flusso (cancellando 1 unità che v stava per inviare al pozzo).

3. Complessità Temporale Il controllo sulla saturazione dell'arco richiede tempo $O(1)$. Le ricerche dei cammini in G_f avvengono tramite visite BFS o DFS. Poiché il testo garantisce che il flusso originale f è aciclico, sappiamo per certo che nel Caso 2 (fase 3) esisteranno sempre dei percorsi a ritroso validi verso s e da t . Ogni singola visita esplora al massimo $|V|$ vertici e $|E|$ archi, richiedendo tempo $O(|V| + |E|)$. Nel caso peggiore, l'algoritmo esegue esattamente 3 visite (una per $u \rightarrow v$, una per $u \rightarrow s$, una per $t \rightarrow v$). Il tempo totale è quindi limitato da $3 \cdot O(|V| + |E|) = O(|V| + |E|)$.

Problem 2: Dynamic Programming (Rome to Paris)

1. Recurrence Relation Let $C(i)$ be the minimum cost to travel the first i miles. To simplify the boundaries, we define the cost of the starting point (Rome) and the destination (Paris) as 0: $A[0] = 0$ and $A[n] = 0$. The base case is: $C(0) = 0$

The recurrence relation for $i > 0$ is:

$$C(i) = A[i] + \min_{1 \leq j \leq k, i-j \geq 0} C(i-j)$$

Explanation: The cost to reach hotel i is the fixed cost of staying at hotel i ($A[i]$) plus the minimum cost of reaching any valid previous hotel within the maximum daily driving distance k .

2. Pseudocode

```

Function MinHotelCost(A, n, k):
    A[0] = 0
    A[n] = 0

    C = array of size n+1 initialized to +infinity
    C[0] = 0

    For i from 1 to n:
        For j from 1 to k:
            If i - j >= 0:
                If C[i-j] + A[i] < C[i]:
                    C[i] = C[i-j] + A[i]

    Return C[n]

```

3. Time Complexity The algorithm uses two nested loops. The outer loop runs n times. The inner loop runs at most k times for each i . Inside the inner loop, all operations (addition, comparison, assignment) take $O(1)$ time. Therefore, the overall time complexity is $O(nk)$.

Problem 3: NP-Completeness (Class Scheduling)

1. Appartenenza a NP (Class scheduling in NP)

- **Witness (Certificato):** Un array t di dimensione n , dove l'elemento i -esimo rappresenta l'ID della stanza (da 1 a k) assegnata alla classe i . La dimensione del certificato è polinomiale limitata.
- **Verificatore:** Un algoritmo che controlla ogni coppia di classi (i, j) per verificare se $t[i] = t[j]$ (ossia, se condividono la stessa stanza). In tal caso, verifica che non ci sia intersezione tra i loro time slot S_i e S_j . Poiché ci sono al massimo $O(n^2)$ coppie da controllare, la verifica avviene interamente in tempo polinomiale.

2. NP-Hardness (Riduzione k-coloring -> Class scheduling) Data un'istanza di k-coloring con un grafo $G = (V, E)$ e un intero k , costruiamo un'istanza di Class scheduling nel seguente modo:

- **Classi e Stanze:** Le $|V|$ classi corrispondono ai vertici del grafo. Le k stanze corrispondono ai k colori disponibili.
- **Time Slots:** Per ogni singolo arco $e = (u, v) \in E$, creiamo un istante temporale univoco t_e . Aggiungiamo questo specifico istante t_e ai time slot di entrambe le classi associate, S_u e S_v .

Dimostrazione costruttiva:

- ($\implies$) Se il grafo G possiede una k -colorazione valida, nessun arco connette due nodi dello stesso colore. Assegnando le classi alle stanze in base al colore dei rispettivi vertici, due classi nella stessa stanza non divideranno mai un istante temporale t_e (il quale esiste solo se c'è un arco tra di loro). Di conseguenza, non c'è alcuna sovrapposizione e il Class scheduling è risolto.
- ($\impliedby$) Se le classi possono essere assegnate a k stanze senza alcuna sovrapposizione temporale, significa che due classi nella stessa stanza non hanno mai istanti t_e in comune. Questo implica che nel grafo originale non esiste l'arco e tra quei due nodi. Sostituendo le stanze con i colori, otteniamo una k -colorazione perfettamente valida per il grafo G .